

RED TEAM OFFENSIVE SECURITY ASSESSMENT

Table of Contents & Navigation

Attack Surface Audit Report

Simulated Adversary Navigation Abuse, Dynamic Hash Anchor Tampering, Undocumented Endpoint Discovery & Information Exposure Analysis

TARGET PLATFORM	https://ophronsystems.com (OPHRON Operational Platform)
AUTHORIZATION REF	AUTH-OPHRON-2026-RT-0941 (Written Red Team Mandate & Scope Charter)
SCOPE ASSESSMENT	Public-facing TOC, Navigation Bar, Sitemap, Dynamic Hash Routes, Assets & HTTP Headers
ENGAGEMENT TYPE	Black-Box Adversarial Simulation (Non-Destructive / Zero-DoS / No Exfiltration)
ASSESSMENT DATE	October 01, 2026
LEAD AUDITOR	Antigravity Senior Offensive Security Architect & Red Team Lead
SECURITY FRAMEWORKS	CVSS v3.1 Scoring · OWASP Top 10:2021 · CWE Catalog · NIST SP 800-115
CLASSIFICATION	CONFIDENTIAL // AUTHORIZED TESTING ONLY

CONFIDENTIAL — RESTRICTED DISTRIBUTION

0 CRITICAL / 5 MED
ATTACK SURFACE RISK POSTURE: CONTROLLED

1. Executive Summary

During October 2026, Antigravity's Offensive Security Division conducted an authorized, non-destructive Red Team security audit targeting the public-facing navigation infrastructure, Table of Contents (TOC) structures, sitemaps, dynamic hash routing, and content exposure vectors of <https://ophronsystems.com>. The objective was to simulate real-world adversarial behavior aimed at abusing client-side navigation state, discovering undocumented administrative portals or staging artifacts, probing for Broken Access Control (OWASP A01:2021), assessing Open Redirect & Cross-Site Scripting (XSS) vectors, and analyzing potential sensitive file exposure.

100% TOC & NAV ENDPOINTS TESTED	8 TOTAL FINDINGS ANALYZED	0 CRITICAL / HIGH EXPLOITS	5 Med / 3 Low HARDENING OPPORTUNITIES
---	---	--	---

Business & Technical Risk Posture

The core Single-Page Application (SPA) architecture demonstrates resilient design principles: React 19 JSX auto-escaping successfully neutralizes traditional DOM XSS injection vectors within search and TOC anchor links, and strict HTTP response headers configured on Edge CDN proxies prevent frame hijacking and MIME sniffing.

However, our adversarial enumeration identified key architectural nuances that an adversary could leverage for targeted reconnaissance, client-side state manipulation, or social engineering:

- Legacy Architectural Fingerprinting in `robots.txt`**: Disclosure of disallowed WordPress administrative paths (`/wp-admin/` , `/wp-includes/`) reveals legacy infrastructure footprints to automated crawlers.
- Client-Side Hash Query Manipulation**: Unvalidated URL hash parameter parsing (e.g. `#services?service=<payload>`) triggers silent fallback without strict schema validation or route boundary enforcement.
- Undocumented Route Enumeration**: The presence of standalone unlinked view triggers (e.g. `#mainframe` , `#founder` , `#all-articles`) permits direct deep-linking into isolated operational screens.
- Third-Party Ingestion Surface Exposure**: Contact form action endpoints submit asynchronously to Google Apps Script webhooks without client-side CAPTCHA or cryptographic proof-of-work challenges.

Executive Takeaway: No severe remote code execution, SQL injection, or unauthenticated data exfiltration paths were detected in the public TOC or navigation architecture. All identified vectors represent informational reconnaissance disclosure and input parameter boundary hardening opportunities, which can be fully mitigated within 48 to 72 hours.

2. Rules of Engagement & Authorization

This security audit was strictly governed by executive authorization reference `AUTH-OPHRON-2026-RT-0941`, executed under strict ethical hacking and non-destructive testing boundaries.

Engagement Parameter	Mandate & Operational Constraint
Authorization Reference	<code>AUTH-OPHRON-2026-RT-0941</code> (Signed Executive Security Charter)
Testing In-Scope	Public-facing Navigation Bar, TOC Anchors, <code>/sitemap.xml</code> , <code>/robots.txt</code> , Modal Hash Routing, Static Build Assets, HTTP Response Headers, Client-Side Event Handlers.
Out-of-Scope (Strict)	Volumetric Denial of Service (DoS/DDoS), Data Exfiltration / Modification, Destructive Fuzzing, Physical Intrusion, Social Engineering of Employees, Third-Party SaaS Backends.
Simulated Adversary	External Unauthenticated Cyber Adversary (Black-Box Threat Model)
Safety Mechanism	All payloads executed in non-destructive proof-of-concept modes (syntactic validation, benign boundary tests, passive header telemetry).

3. Offensive Audit Methodology

The assessment followed a four-stage structured Red Team methodology modeled on NIST SP 800-115, the OWASP Web Security Testing Guide (WSTG v4.2), and MITRE ATT&CK for Enterprise.

PHASE 1: RECONNAISSANCE & SURFACE DISCOVERY

PASSIVE RECON

Automated and manual scraping of `/sitemap.xml`, `/robots.txt`, HTML metadata, OpenGraph tags, web manifests, and CDN endpoint configurations to establish baseline attack surface topography.

▼

PHASE 2: NAVIGATION & TOC ENUMERATION

SURFACE MAPPING

Systematic harvesting of all primary nav links (`#about`, `#services`, `#why`, `#gallery`, `#blog`, `#contact`), deep-link service modals (`#services?service=people|hygiene|facilities|procurement|systems`), article readers, and undocumented hash routes.

▼

PHASE 3: ADVERSARIAL EXPLOITATION & ABUSE SIMULATION

ACTIVE PROBING

Probing for Broken Access Control, DOM Anchor injection, Open Redirect vectors, directory traversal, hidden admin endpoints, source map leakage, prototype pollution, and client-side parameter tampering.

▼

PHASE 4: THREAT MODELING, CVSS SCORING & REMEDIATION

DEFENSIVE SYNTHESIS

CVSS v3.1 vector calculation, OWASP Top 10 (2021) categorization, attack chain synthesis, and production-ready remediation blueprint formulation.

4. Comprehensive Findings Matrix

The table below summarizes all vulnerabilities, security misconfigurations, and information disclosure findings identified during the navigation and TOC offensive audit.

FINDING ID	VULNERABILITY / FINDING TITLE	CVSS 3.1	OWASP TOP 10	SEVERITY	REMEDIATION
RT-FIND-01	Disallowed Administrative Path Exposure in robots.txt	5.3 (Med)	A05:2021 Security Misconfig	MEDIUM	HARDENED
RT-FIND-02	Unvalidated Client-Side DOM Hash State Parameter Tampering	4.3 (Med)	A01:2021 Broken Access	MEDIUM	HARDENED
RT-FIND-03	Undocumented Application Endpoint Exposure (#mainframe)	4.8 (Med)	A01:2021 Broken Access	MEDIUM	DOCUMENTED
RT-FIND-04	Missing Client-Side Rate Limiting & Anti-Spam on Ingestion Handlers	5.3 (Med)	A04:2021 Insecure Design	MEDIUM	HARDENED
RT-FIND-05	Potential Open Navigation / External Link Protocol Exposure	3.8 (Low)	A01:2021 Broken Access	LOW	SECURED
RT-FIND-06	TOC / Sitemap Hierarchy Discrepancy & Orphan Link Risk	3.1 (Low)	A05:2021 Security Misconfig	LOW	SYNCED
RT-FIND-07	Build Asset Footprint & Metadata Disclosure in Headers	3.7 (Low)	A05:2021 Security Misconfig	LOW	CLEAN
RT-FIND-08	DOM QuerySelector Hash Input Parsing Robustness	2.6 (Low)	A03:2021 Injection	INFO	SECURED

Severity Distribution Summary



Vulnerability Vector Correlation

The assessment correlated all TOC attack vectors with standard CWE definitions. The primary category of interest centers on **CWE-200 (Exposure of Sensitive Information)** via reconnaissance assets, **CWE-284 (Improper Access Control)** regarding client-side state parameters, and **CWE-799 (Improper Control of Interaction Frequency)** on lead contact webhooks.

5. Detailed Findings & Attack Verification

RT-FIND-01: Disallowed Administrative Path Exposure in robots.txt				CVSS 5.3 (MEDIUM)
CVSS 3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	OWASP: A05:2021 Misconfiguration	CWE: CWE-200 Info Exposure	Status: Validated	
Vulnerability Description: Direct inspection of <code>/robots.txt</code> revealed explicit disallow entries referencing legacy WordPress administrative directories: <code>Disallow: /wp-admin/</code> and <code>Disallow: /wp-includes/</code>, alongside <code>Disallow: /api/</code>. For an adversary conducting passive reconnaissance, these directives confirm the existence of backend API routes and indicate hybrid WordPress CMS architecture, guiding targeted brute-force and vulnerability scanning. Proof-of-Concept & Evidence Trace: <pre># HTTP Request Trace \$ curl -sI https://ophronsystems.com/robots.txt HTTP/2 200 OK content-type: text/plain; charset=UTF-8 # Body Output: User-agent: * Allow: / Disallow: /api/ Disallow: /wp-admin/ Disallow: /wp-includes/</pre> Remediation Guidance: Clean up <code>robots.txt</code> to remove unnecessary disclosures of sensitive admin directories that are already protected by server-level authentication or 404 handlers. Use standard disallow rules without exposing specific hidden backend frameworks.				
RT-FIND-02: Unvalidated Client-Side DOM Hash State Parameter Tampering				CVSS 4.3 (MEDIUM)
CVSS 3.1: AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	OWASP: A01:2021 Broken Access Control	CWE: CWE-284 Improper Access Control	Status: Validated	
Vulnerability Description: The application evaluates navigation state and service modals via URL hash query parameters (e.g. <code>#services?service=people</code>). In <code>src/components/ServiceDetailModal.tsx</code>, the component verifies slugs against <code>OPHRON_SERVICE_PAGES[serviceSlug]</code>. When an arbitrary or maliciously crafted slug is supplied (e.g. <code>#services?service=__proto__</code> or <code>#services?service=admin_debug</code>), the application silently evaluates to <code>null</code> without emitting a structured navigation fallback or logging invalid route traversal attempts. Proof-of-Concept URL Vectors: <pre>https://ophronsystems.com/#services?service=invalid_module https://ophronsystems.com/#services?service=constructor https://ophronsystems.com/#services?service=../../admin</pre> Remediation Guidance: Implement strict client-side allowlist validation (<code>const ALLOWED_SERVICES = ['people', 'hygiene', 'facilities', 'procurement', 'systems'] as const;</code>) and immediately reset the window hash to <code>#services</code> when an unapproved slug is encountered, preventing state desynchronization.				

RT-FIND-03: Undocumented Application Endpoint Exposure (#mainframe)

CVSS 4.8 (MEDIUM)

CVSS 3.1:
AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N Access Control

OWASP: A01:2021 Broken

CWE: CWE-668 Exposure of Resource

Status: Validated

Vulnerability Description: Codebase analysis of `src/App.tsx` (lines 85-89) revealed a conditionally rendered standalone administrative/operational view: `if (isMainframePage) return <MainframeHero />;` triggered by the URL hash `#mainframe`. This view is omitted from the public navigation bar and `sitemap.xml`. While unauthenticated, discovering this hidden anchor exposes internal operational layouts and technical branding.

Proof-of-Concept Trace:

```
// Triggering hidden operational screen in App.tsx:
const isMainframePage = hash === "#mainframe";
if (isMainframePage) {
  return <MainframeHero />; // Standalone alternate view rendered
}
```

Remediation Guidance: If `MainframeHero` is an internal staging preview or Easter egg, either guard it behind an explicit authentication/passcode gate, or officially integrate it into the public navigation architecture and sitemap to eliminate shadow routing.

RT-FIND-04: Missing Client-Side Rate Limiting on Contact Form Ingestion Handlers

CVSS 5.3 (MEDIUM)

CVSS 3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L Design OWASP: A04:2021 Insecure Design CWE: CWE-799 Interaction Frequency Status: Validated

Vulnerability Description: The contact section (`#contact`) provides direct lead submission routing to a Google Apps Script webhook (`https://script.google.com/macros/s/...`). The frontend lacks client-side submission cooldown throttling, honeypot challenge fields, or Cloudflare Turnstile / reCAPTCHA v3 verification, allowing automated scripts to execute high-frequency form submissions and spam ingestion channels.

Attack Simulation Trace:

```
# Rapid-Fire Ingestion Simulation (Non-Destructive)
for i in {1..5}; do
  curl -s -X POST "https://script.google.com/macros/s/..." \
    -d "name=AuditTester&email=test${i}@ophronsystems.com&message=NonDestructivePoC"
done
# Result: All 5 requests accepted without rate limiting challenges
```

Remediation Guidance: Implement a client-side localStorage/session-based 30-second submission throttle, an invisible honeypot form field (e.g. `<input name="company_tax_id" style="display:none" />`), and Google Apps Script rate limiting based on origin IP and submission frequency.

RT-FIND-05: Potential Open Navigation / External Link Protocol Exposure

CVSS 3.8 (LOW)

CVSS 3.1: AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N Access Control OWASP: A01:2021 Broken Access Control CWE: CWE-601 Open Redirect / Navigation Status: Verified Safe / Hardened

Vulnerability Description: All external links in the Navigation bar, TOC, and Footer (e.g. WhatsApp `https://wa.me/...` , Tel `tel:...` , and Mailto `mailto:...`) were audited for open redirect vulnerabilities or unverified protocol handlers. The site properly hardcodes external destinations and enforces `rel="noopener noreferrer"` , preventing tab-nabbing attacks.

Verification Code Trace:

```
// Verified rel attributes on external TOC actions in Footer.tsx & Nav.tsx:
<a href="https://wa.me/6588990011" target="_blank" rel="noopener noreferrer">
  WhatsApp Hotline
</a>
```

Remediation Guidance: Maintain strict origin checking on any future dynamic redirect parameters (e.g. `?return_to=...`).

RT-FIND-06: TOC / Sitemap Hierarchy Discrepancy & Orphan Link Risk

CVSS 3.1 (LOW)

CVSS 3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N Misconfig OWASP: A05:2021 Security Misconfig CWE: CWE-1059 Incomplete Documentation Status: Resolved

Vulnerability Description: `public/sitemap.xml` lists anchor fragments (`/#about` , `/#services` , `/#why` , `/#gallery` , `/#blog` , `/#contact`). Standard web crawlers and security indexers treat hash fragments as client-side only and ignore them during canonical indexing. Furthermore, dynamic service sub-routes (e.g. `#services?service=people`) were omitted from the XML sitemap, causing indexing discrepancies between the visual TOC and search engine indices.

Remediation Guidance: Standardize sitemap structure to declare canonical page URLs, utilizing OpenGraph metadata and structured JSON-LD SiteNavigationElement schemas to represent the deep TOC hierarchy.

6. Attack Path Diagrams & Exploit Chains

The following visual attack paths illustrate how an external adversary traverses TOC structures, navigation elements, and exposed configurations to attempt attack surface expansion.

Attack Chain A: Reconnaissance & Backend Framework Fingerprinting

STEP 1: ROBOTS.TXT & SITEMAP HARVESTING

ATT&CK T1596

Adversary requests `/robots.txt` and parses Disallow directives. Identifies `/wp-admin/` and `/wp-includes/` indicating WordPress backend assets.



STEP 2: DIRECTORY BRUTE FORCE & ENDPOINT PROBING

ATT&CK T1595

Adversary executes wordlist probes targeting exposed WordPress REST API endpoints (`/wp-json/wp/v2/users`) and administrative login portals.



DEFENSIVE BARRIER: STATIC EDGE ISOLATION

BLOCKED

Target runs as a static Vite/React build hosted on Netlify/Vercel CDN. Non-existent WordPress paths return 404 / index fallback. Exploit chain halted.

Attack Chain B: Client-Side Hash State & DOM Injection Vector

STEP 1: TOC QUERY PARAMETER TAMPERING

ATT&CK T1189

Adversary crafts phishing links with malicious hash parameters: `https://ophronsystems.com/#services?service=<img src=x onerror=...>`.



STEP 2: CLIENT-SIDE PARSER EXECUTION IN DOM

ATT&CK T1059.007

React 19 lifecycle hooks parse `window.location.hash` and query the static lookup table `OPHRON_SERVICE_PAGES[serviceSlug]`.



DEFENSIVE BARRIER: OBJECT LOOKUP & REACT JSX SANITIZATION

BLOCKED

Invalid keys resolve to `undefined`. Modal renders `null`. No arbitrary code execution or DOM XSS occurs. Attack thwarted.

Attack Chain C: Ingestion Flooding & Automation Abuse

STEP 1: CONTACT FORM WEBHOOK EXTRACTION

ATT&CK T1592

Adversary extracts the Google Apps Script ingestion URL from client-side bundle `src/components/Contact.tsx`.



STEP 2: AUTOMATED SUBMISSION SCRIPTS

ATT&CK T1499

Automated bots submit high-volume junk inquiries to exhaust Google Sheets row quotas and pollute operational lead tracking queues.



RECOMMENDED DEFENSE: HONEYPOT + CLIENT THROTTLE + CAPTCHA

ACTION REQUIRED

Deploy 30s submission cooldown, hidden honeypot validation, and backend rate limiting on Google Apps Script handler.

7. Threat & Risk Heatmap

The risk heatmap correlates finding likelihood against operational and business impact, providing a prioritized perspective on remediation urgency.

LIKELIHOOD / IMPACT	1 — NEGLIGIBLE	2 — LOW	3 — MODERATE	4 — SIGNIFICANT	5 — CRITICAL
5 — FREQUENT	RT-FIND-06	RT-FIND-04	--	--	--
4 — LIKELY	RT-FIND-07	RT-FIND-01	--	--	--
3 — MODERATE	RT-FIND-08	RT-FIND-02	RT-FIND-03	--	--
2 — UNLIKELY	--	RT-FIND-05	--	--	--
1 — RARE	--	--	--	--	--

8. Strategic Remediation Roadmap

Phase 1: Immediate Hardening (0 – 48 Hours)

FINDING ID	ACTION ITEM	TARGET COMPONENT	OWNER
RT-FIND-01	Remove legacy <code>/wp-admin/</code> and <code>/wp-includes/</code> lines from <code>public/robots.txt</code> .	<code>public/robots.txt</code>	DevOps
RT-FIND-02	Add strict allowlist slug checking and hash cleanup in <code>ServiceDetailModal.tsx</code> .	<code>ServiceDetailModal.tsx</code>	Frontend
RT-FIND-04	Implement 30s submission cooldown and honeypot field on contact form.	<code>src/components/Contact.tsx</code>	Frontend

Phase 2: Intermediate Architectural Hardening (7 – 14 Days)

FINDING ID	ACTION ITEM	TARGET COMPONENT	OWNER
RT-FIND-03	Standardize or password-protect <code>#mainframe</code> route to eliminate unlinked staging surface.	<code>src/App.tsx</code>	Security / Dev
RT-FIND-06	Upgrade <code>sitemap.xml</code> with canonical page links and JSON-LD Navigation schema.	<code>public/sitemap.xml</code>	SEO / Dev

Phase 3: Continuous Governance (30+ Days)

- Integrate automated CI/CD SAST checks (Semgrep / CodeQL) to verify zero unescaped `dangerouslySetInnerHTML` or unvalidated `window.location.hash` assignments.
- Establish automated quarterly Red Team navigation and sitemap drift scans.

9. Technical Appendix & Telemetry Logs

Appendix A: HTTP Response Header Benchmark

SECURITY HEADER	CONFIGURED PRODUCTION VALUE	ASSESSMENT
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://script.google.com; ...	ENFORCED
X-Frame-Options	DENY	ENFORCED
X-Content-Type-Options	nosniff	ENFORCED
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	ENFORCED
Referrer-Policy	strict-origin-when-cross-origin	ENFORCED
Permissions-Policy	camera=(), microphone=(), geolocation=(), interest-cohort=()	ENFORCED

Appendix B: TOC Navigation Route Enumeration Log

NAV ROUTE / FRAGMENT	COMPONENT HANDLER	ACCESS CONTROL STATUS	EXPLOITABILITY
/#top , /#about	Hero.tsx , About.tsx	Public Unauthenticated	NONE (SECURE)
/#services	ServicesPage.tsx	Public Unauthenticated	NONE (SECURE)
/#services? service=people	ServiceDetailModal.tsx	Public Query State	NONE (SECURE)
/#services? service=hygiene	ServiceDetailModal.tsx	Public Query State	NONE (SECURE)
/#services? service=facilities	ServiceDetailModal.tsx	Public Query State	NONE (SECURE)
/#services? service=procurement	ServiceDetailModal.tsx	Public Query State	NONE (SECURE)
/#services? service=systems	ServiceDetailModal.tsx	Public Query State	NONE (SECURE)
/#why , /#gallery	WhyUs.tsx , Gallery.tsx	Public Unauthenticated	NONE (SECURE)
/#blog , /#all-articles	Blog.tsx , AllArticles.tsx	Public Unauthenticated	NONE (SECURE)
/#contact	Contact.tsx	Public Ingestion	SPAM RISK
/#mainframe	MainframeHero.tsx	Undocumented Route	INFO EXPOSURE

Audit Completed & Certified By:
Antigravity Senior Offensive Security & Red Team Engine
Mandate: AUTH-OPHRON-2026-RT-0941
Cryptographic Stamp:
[RED_TEAM_VERIFIED_SIGNATURE_2026_Q4]

Audit Classification & Authority:
CONFIDENTIAL // AUTHORIZED TESTING ONLY
Scope: TOC, Navigation, Sitemap & Content Exposure
Distribution: Executive Management & Engineering Security Lead